

CS-471L: Compiler Construction Lab

Mini Compiler Project (Integrated Lab Modules)

Spring 2026

University of Engineering and Technology, Lahore

Department of Computer Science

Instructor: aliya.farooq@uet.edu.pk

Reference Book: Compilers, Principles, Techniques, and Tools (Aho, Lam, Sethi, Ullman)

1. Project Overview

This document describes the final lab project for the Compiler Construction course. In this project, every group will combine all the lab modules that were built during the semester into one working mini compiler. The lab tasks that have been uploaded on Google Classroom form the base of this project and must be reused.

Three more topics are to be added to the mini compiler: LR Parser, Symbol Table Manager, and Error Handling. The LR Parser and the Symbol Table Manager are required modules. The Error Handler is a bonus task and carries extra marks.

Each group will work with the language subset and the CFG grammar that was assigned to them during the lab tasks. The same grammar must be used in every module of the mini compiler so that the lexer, parsers, symbol table, and error handler all agree on the same source language.

2. Group Formation

- Minimum group size: 2 students.
- Maximum group size: 3 students.
- Each group must register its members at the start of the project and pick one member as the group lead.
- The modules must be divided fairly among the members.
- Every member must attend the demo and must be able to explain the module they built.
- Marks may differ within a group based on the individual viva.

3. Language Subset and Grammar

Each group will use the language subset and the BNF grammar that was assigned to them during the lab sessions. Different groups have different grammars (for example a subset of Pascal, MicroJava, Decaf Language). The chosen grammar must be used consistently across the lexical analyzer, the parsers, the symbol table, and the error handler.

The grammar must be supplied in BNF form. If the grammar contains left recursion or is not left factored, it must be transformed before being used with the recursive descent and the predictive parsers. The original (left-recursive) grammar may be used with the LR parser.

4. Compiler Modules

The mini compiler must contain the following modules, all linked together into one program.

4.1 Lexical Analyzer (Lab 3)

- Read the source program character by character using double buffering (from Lab 2).
- Produce a token stream as a sequence of (token, attribute) pairs.
- Track the line number and column number of every token.
- Skip whitespace and comments.
- Recognize identifiers, keywords, numbers, character constants, operators, and punctuation as defined by the assigned grammar.
- Pass one token at a time to the parser when the parser asks for it.

4.2 Recursive Descent Parser (Lab 4)

- One parsing routine per non-terminal of the grammar.
- Each routine calls the lexical analyzer to read the next token.
- A syntax error is reported when the input does not match the expected grammar rule.
- The grammar used here must be free of left recursion and must be left factored.

4.3 Non-Recursive Predictive Parser (Lab 5)

- Use an LL(1) parsing table built from the FIRST and FOLLOW sets of the grammar.
- Use an explicit stack instead of recursive function calls.
- Should produce the same parse result as the recursive descent parser for the same input.
- Print the parsing table and the FIRST and FOLLOW sets in the project report.

4.4 LR Parser (New Module)

- Construct the LR parsing table for the assigned grammar.
- Any of LR(0), SLR(1), LALR(1), or canonical LR(1) may be used. The choice must be justified in the report.
- Implement the standard shift-reduce algorithm with a stack of states and a stack of grammar symbols.
- Print the action table and the goto table in the project report.
- For at least one sample input, show a trace with the stack contents, the remaining input, and the action taken at each step.

4.5 Symbol Table Manager (Lab 7, Extended)

- Use a hash-based table for fast lookup.
- Support insert, lookup, and delete operations.
- Store the name, kind (variable / constant / function / array / class), type, scope level, and source line number for each entry.
- Handle nested scopes using a stack of symbol tables or a linked scope chain.
- The parser must call the symbol table at every declaration and at every use of a name.
- The contents of the symbol table must be printable at any point for debugging.

4.6 Error Handler (Bonus Task)

- Detect and report lexical, syntactic, and semantic errors.

- Print the line number, column number, and a short message for every error.
- Use panic-mode recovery so that the parser does not stop at the first error.
- For the LR parser, use the error entries in the parsing table to invoke the recovery routine.
- For the predictive parser, use synchronization sets based on FOLLOW sets to skip tokens after an error.
- Print a summary of all detected errors at the end of compilation.
- Extra marks are given for phrase-level recovery and for clear, helpful error messages.

5. Deliverables (Updated)

The deliverables for the mini compiler project are different from the earlier milestones. Each group must submit the following four items.

Deliverable	Description	Submission Week	Weight
D1: Group Sheet and Grammar	Group members with roll numbers, the assigned grammar in BNF, FIRST and FOLLOW sets, LL(1) parsing table, LR parsing table (action + goto), and at least three sample programs.	Week 13	10%
D2: Integrated Source Code	All source files for the lexer, three parsers, symbol table, and error handler. Build script (Makefile or batch file). README file. Folder with sample input files. Folder with output of every module.	Week 16	40%
D3: Project Report	Grammar specification, description of each module, construction of the LR parsing table (showing the working), discussion of how the modules are integrated, sample input and output, limitations, and references.	Week 16	25%
D4: Live Demonstration	Fifteen-minute demo per group. Run sample programs through every module. Each member explains their part. Short Q&A session at the end.	Week 16	25%

The four deliverables together make up the project marks. The grading rubric in Section 6 is applied on top of these deliverables.

6. Grading Rubric (Unchanged)

The grading rubric remains the same as the one given in the original semester plan.

Component	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Poor (<50%)
Lexer	All tokens, robust errors	Most tokens, basic errors	Basic tokens work	Major issues

Component	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Poor (<50%)
Parser	Complete grammar, AST	Most grammar, some AST	Basic statements	Incomplete
Symbol Table	Scopes, efficient	Basic scopes	Works	Not functional
Semantics	Type checking, validation	Basic type checking	Minimal	Missing
Integration	Seamless, well-tested	Works together	Basic integration	Poor
Documentation	Professional, complete	Good coverage	Adequate	Insufficient
Demo	Confident, handles edge cases	Good presentation	Basic demo	Unprepared

Bonus Marks (Up to 15%)

- Working error handler with both panic-mode and phrase-level recovery.
- Clear and friendly error messages with line and column information.
- LR parsing table built by hand and matched with an auto-generated table from a tool such as YACC or Bison.
- AST visualization (text or graphical).

7. Submission Guidelines

File Naming

CS471L_Project_Group#_RollNo.zip

Example: CS471L_Project_Group3_2021CS101.zip

Folder Layout

submission/

```

├─ src/                (all source code files)
├─ docs/               (project report and grammar)
├─ test/               (sample programs)
├─ output/             (sample outputs for each module)
├─ Makefile            (build file)
└─ README.md           (build and run instructions)
```

Submission Platform

Upload on the GCR only. Email submissions will not be accepted unless the GCR is down.

Late Submission Policy

Days Late	Penalty
1 day	−10%
2 days	−20%
3 days	−30%
More than 3 days	Not accepted

8. Academic Integrity

- Code sharing between groups is not allowed.
- Code from previous semesters or from public online repositories is not allowed.
- AI tools may be used to understand concepts only, not to generate the source code of the compiler.
- All submissions will be checked with MOSS (Measure of Software Similarity).
- Plagiarism penalties listed in the course policy apply.

9. Evaluation Format

- Each group will be evaluated in a separate slot in the final lab week.
- The teaching assistant will run a few unseen test programs through the compiler.
- Every member will be asked at least two questions on the module they built.
- Every member will be asked at least one question on how the modules are integrated.
- Marks may differ within a group based on the individual viva.
- Bring a laptop with the compiler ready to run. Code that does not compile at demo time will lose integration marks.

10. Quick Checklist Before Submission

- All six modules build and run from a single command.
- Lexer prints the token stream for the sample input.
- Recursive descent parser, predictive parser, and LR parser all accept the same valid input and reject the same invalid input.
- Symbol table dump is correct after declarations.
- Error handler reports line and column numbers and recovers from at least one error per input file.
- Project report contains the grammar, FIRST/FOLLOW sets, LL(1) table, LR action and goto tables, and sample traces.
- All members can explain their part of the code.

End of Document